

Write the Python program to implement Minimax Algorithm for Gaming

Aim

To implement the Minimax algorithm for game decision making.

Objective

To find the optimal move by maximizing the player's score and minimizing the opponent's score.

Theory

The **Minimax Algorithm** is a decision-making algorithm used in Artificial Intelligence for two-player, turn-based games such as Tic Tac Toe, Chess, and Checkers. It assumes that one player tries to maximize the score while the opponent tries to minimize it. The algorithm recursively explores all possible game states, evaluates the leaf nodes, and selects the move that gives the best possible outcome for the maximizing player while considering the opponent's optimal moves.

Algorithm

1. Create a game tree.
2. Traverse recursively.
3. If leaf node, return its value.
4. If maximizing player, choose maximum value.
5. If minimizing player, choose minimum value.
6. Return optimal value.

Flowchart

Start

|

Create Game Tree

|

Leaf Node?

|

└─ Yes ─┘

| |

Return Explore Children

Value

|

Max Player?

|

Choose Maximum

|

Else Choose Minimum

|

Return Best Value

|

Stop

Python Code

```
import math
```

```
def minimax(depth, node, isMax, values, height):
```

```
    if depth == height:
```

```
        return values[node]
```

```
    if isMax:
```

```
        return max(
```

```
            minimax(depth + 1, node * 2, False, values, height),
```

```
            minimax(depth + 1, node * 2 + 1, False, values, height)
```

```
        )
```

```
    else:
```

```
        return min(
```

```
            minimax(depth + 1, node * 2, True, values, height),
```

```
        minimax(depth + 1, node * 2 + 1, True, values, height)
    )
```

```
values = [3, 5, 6, 9, 1, 2, 0, -1]
```

```
height = 3
```

```
print("Optimal Value:", minimax(0, 0, True, values, height))
```

Sample Output

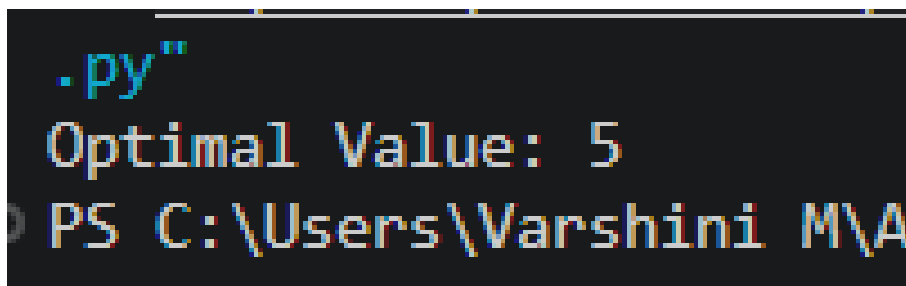
Optimal Value: 5

Result

The Minimax algorithm successfully found the optimal game value.

Conclusion

The Minimax algorithm helps in selecting the best possible move by considering both maximizing and minimizing players.



```
.py"
Optimal Value: 5
PS C:\Users\Varshini M\A
```

C: > Users > Varshini M > OneDrive > Desktop > Here!!! > ARTIFICIAL INTELLIGENCE > VSC EXPs > EXP1.p

```
1  import math
2
3  def minimax(depth, node, isMax, values, height):
4
5      if depth == height:
6          return values[node]
7
8      if isMax:
9          return max(
10             minimax(depth + 1, node * 2, False, values, height),
11             minimax(depth + 1, node * 2 + 1, False, values, height)
12         )
13
14     else:
15         return min(
16             minimax(depth + 1, node * 2, True, values, height),
17             minimax(depth + 1, node * 2 + 1, True, values, height)
18         )
19
20 values = [3, 5, 6, 9, 1, 2, 0, -1]
21 height = 3
22
23 print("Optimal Value:", minimax(0, 0, True, values, height))
24
```